Out of the Model Context, Into the Program: An Architecture for Deep-Search Agents

Islam Torky
neuland.ai AG
Cologne, Germany
Paderborn University
Paderborn, Germany

Ismail Ben Ayed
Technical University of Munich
Munich, Germany

Harshal Nandigramwar
neuland.ai AG
Cologne, Germany

Syed Qutub
neuland.ai AG
Cologne, Germany

Gehad Dahami
neuland.ai AG
Cologne, Germany

Dietlind Zühlke
TH Köln
University of Applied Sciences
Cologne, Germany

N'Dah Jean Kouagou
Data Science Group (DICE)
Paderborn University
Paderborn, Germany

Axel-Cyrille Ngonga Ngomo
Data Science Group (DICE)
Paderborn University
Paderborn, Germany

***Abstract*—Deep-search agents answer hard questions by searching a corpus many times and reasoning over what they find; a single vector search or retrieval-augmented generation step rarely suffices, because the evidence is spread over documents that no one query returns. In the standard design every search result is appended to the prompt, so the agent either sees only a fragment of each document or reasons over an ever-growing prompt. We propose an architecture that moves retrieved text out of the prompt and into the program. Whole documents are held as variables in a Python read-eval-print loop (REPL), the model reads them by writing code, and only what the code prints enters its context. The REPL itself is not new: we reuse it from recursive language models and agents such as Agentica. A search cap, not the context window, bounds how much the agent retrieves, so it can fetch whole documents and search wide. We compare the architecture with the benchmark's function-calling agent and variants of it that tackle the same trade-off inside the prompt, with one model for every agent, and ablate its components one at a time. On BrowseComp-Plus questions held out from development it answers 87% versus 78% both for the function-calling agent with a whole-document tool and for Fetch-then-Explore, a recent agent that keeps fetched documents in a workspace, with about a quarter of Fetch-then-Explore's tokens; on the development questions it also uses about a quarter of the function-calling agent's tokens per question. The ablations show where the gain comes from: wide searches find the evidence, and keeping documents out of the prompt is what makes such searches affordable. It also leads on FRAMES and runs with an open-weight model.**

***Index Terms*—Deep-search agents, retrieval-augmented generation, recursive language models, context engineering, agent evaluation**

I. Introduction

Deep-search agents answer a question by searching a corpus repeatedly and reasoning over what the searches return [1]. A hard question combines several constraints whose evidence is spread over several documents, so a single retrieval step, as in standard retrieval-augmented generation, rarely suffices: on FRAMES, one BM25 retrieval lifts Gemini 1.5 Pro only from 41 to 47%, and multi-step retrieval to 66% [2]. The agent needs many searches and many documents before it can answer. In the prevailing design, from the reason-and-act loop [3], every search result is appended to the conversation [4]. This forces a trade-off. Short snippets keep the prompt small but hide evidence deeper in a page; whole documents expose the evidence, but the prompt grows with every search, and models read long prompts worse even when the needed text is present [5], [6].

Recursive language models (RLMs) [7] show a way out of this trade-off for long inputs. An RLM never places its input in the prompt: the input is a variable in a Python read-eval-print loop (REPL), the model inspects it by writing code, can call itself recursively on parts of it, and sees only what its code prints. An RLM can be given a search tool, and we build one as a baseline, but the natural way to do so keeps the standard design's output: each search prints its snippets into the prompt, and whole documents are read through recursive sub-calls. The prompt still grows with every search, and reading a document costs further model calls.

We propose an architecture that brings the RLM idea to retrieval by moving retrieved text out of the prompt and into the program (Fig. 1).[1] The agent runs in the RLM's REPL, without recursive calls. Each search returns whole documents into a variable that persists across turns; the model reads them by writing code, for example slicing a document or matching a pattern across every held document, and only what its code prints enters the prompt. A search cap, a hard limit on searches per question, replaces the context window as the bound on how much the agent retrieves.

Because a held document costs the prompt nothing until part of it is printed, the agent can afford to search wide: it chooses how many documents each search returns, up to 50, and in our runs asked for about 30. Wide searches find evidence

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

[1] https://github.com/neulandAI/program-state-agents



Fig. 1. Where the results of one search go, for each agent of Section III (purple: our architecture). Top of each card: what one search returns. Grey box: the prompt, which the model reads at every call. Dashed box: program state, which the model reaches only by running code; only what the code prints enters the prompt. Bottom: tokens one search adds to the prompt. The reference agent's document-tool variant also puts whole documents into the prompt.

that narrow ones miss, and selective printing keeps the prompt short enough to read well. The architecture builds on RLMs, on code as the action space [8] and on tool results kept in the execution environment [9]; our contribution is its design for deep search over a retrieved corpus and the evidence for why it works.

We compare it with the standard function-calling agent and with variants of it that tackle the same trade-off inside the prompt: a whole-document tool to expose hidden evidence, a trimmed or summarised history to curb prompt growth, and the same search cap or wider searches to match the search budget. We pin the model, GPT-5 [10], not because the architecture depends on it but so that every agent uses the same model and every ablation changes exactly one thing. BrowseComp-Plus [4] is our controlled testbed, because its gold labels separate finding the evidence from reading it. FRAMES [2] and OBLIQ-Bench [11] test how far the architecture carries. The open-weight Qwen3.6 [12] tests whether it works without a frontier model, a first step towards distilling or fine-tuning the behaviour into smaller models.

We ask three questions:

- **RQ1 (effectiveness).** Does the architecture answer more questions than the standard design and its in-prompt variants?
- **RQ2 (mechanism).** Why does it work: does it find more evidence or read it better, and which component is responsible?
- **RQ3 (generality).** Does it carry over to other benchmarks, retrievers and an open-weight model?

Our contributions are: (1) a program-state architecture for deep-search agents that keeps retrieved documents out of the prompt and bounds retrieval with a search cap, answering more questions than the standard design with fewer tokens than any of its in-prompt variants; (2) a controlled evaluation against the standard design and its in-prompt variants, including questions held out from development, with ablations that isolate search width, where documents live, reading in code and the `grep_docs` tool; (3) design guidance for agent engineers.

II. Related Work

State outside the prompt. Code as an agent's action space [8] and code execution in which intermediate tool results stay in the execution environment [9] are established practice; filtering web results with code before they reach the context raised Claude Opus 4.6 on BrowseComp from 45.3 to 61.6% [13]. RLMs place the material a model works on in a REPL; on a 1,000-document variant of BrowseComp-Plus given as input, an RLM with GPT-5 scored 88.0% without sub-calls against 70.5% for a compaction agent [7]. Closest to us are agents that keep documents as files and read them with shell tools: Codex with GPT-5 answers 88.5% of 200 BrowseComp-Plus questions over the corpus as files [14]; direct corpus interaction (DCI) and Dr-DCI search the corpus or a per-question workspace with shell tools [15], [16]. OpenSaC and Scroll let agents keep search results or tool outputs in program state and print only selected parts [17], [18]. Fetch-then-Explore writes fetched pages to a per-question workspace and reads them with grep and read tools, while search results still enter the prompt [19]; we compare with it directly (Section V-A). Agentica's GPT-5 REPL agent searches 200 results at a time (78.4% on the leaderboard) [20]. RISE varies width only inside large workspaces [21]. Holding results outside the prompt can also hurt: Sen et al. [22] found that delivering grep results as files rather than inline lowered accuracy in nine of ten model–harness pairs on chat histories (in the worst case from 93.1 to 55.2%). State outside the prompt pays only if the model reads it back. Programmatic search agents (PSA) compare a query-only agent with tool and code agents sharing a candidate workspace on BrowseComp-Plus, varying interface, not width [23]. Pi-Serini caches rankings of up to 1,000 BM25 hits outside the context and pages through them (83.1% with GPT-5.5) [24]. We propose the architecture for deep search

and isolate, under equal caps and with gold labels, which of its components carries the gain.

Long prompts and context management. Models degrade as prompts grow even with perfect retrieval [5], [6], [25]. Remedies summarise the history [26] or mask stale tool results; masking helped most models on BrowseComp-Plus, with fetched pages kept reachable in a pool [27].

III. Architecture

We describe three agents, each building on the one before it (Fig. 1). The reference agent keeps everything in the prompt. The snippet REPL agent moves the work into a REPL but still prints what its searches return. The document REPL agent, our architecture, also moves the retrieved documents out of the prompt and into the program.

A. Reference Agent

The *reference agent* is the search agent shipped with BrowseComp-Plus, run unmodified [4]. It uses function calling through the OpenAI Responses API and one `search` tool that returns the five best-ranked documents, each cut to its first 512 tokens (a *snippet*). Five hits of 512 tokens are the benchmark's documented settings; we keep them for every snippet-based agent, so all baselines see the same search results. The agent runs for up to 100 turns and keeps every tool result in the conversation. We also run variants that tackle the trade-off inside the prompt: adding the benchmark's `get_document` tool so the agent can read whole documents in its prompt; holding that agent to our cap of 10 searches, with 5 or 25 snippets per search (it learns the cap when a search is refused and gets no initial search); and keeping the conversation under 30,000 tokens, either by trimming the oldest tool results or by folding them into a summary written by the same model.

B. Snippet REPL Agent

The snippet REPL agent and our document REPL agent (Section III-C) both run in the REPL of the RLM library [7], [28]: in each turn (one model call) the model writes Python, the REPL executes it, and only the printed output is appended to the conversation, cut at 20,000 characters per code block, the library's default, which we did not tune. The model ends by assigning its answer to a reserved variable.

The *snippet REPL agent* reads through a sub-model, as recursive language models do: each search prints the same five snippets the reference agent receives, `get_document` fetches a whole document, and the model passes fetched documents to a sub-model instead of printing them. Its snippets enter the prompt, so its prompt still grows with every search; it has no search cap (we also run it with one) and a turn limit of 100.

C. Document REPL Agent

Our architecture keeps the REPL and changes what a search returns and where it goes. `search(query, k)` returns the complete text of the top k documents and merges them into a dictionary `DOCS`, keyed by document id, that persists across turns; it prints one summary line of about 25 tokens and returns the hits to the calling code. The model chooses k, the *width* of the search, from 1 to 50. `grep_docs` matches a regular expression against every held document and prints the matches in context, following Sen et al., who found literal matching often better than vector retrieval [22]; `unread_docs` lists documents not yet looked at. Neither counts as a search.

A *search cap*, announced in the prompt and enforced outside the model's code, limits searches per question (10, or 20 in the tuned setting). Held documents do not enter the prompt, so how much the agent retrieves is limited by the cap rather than by the context window. Because only printed output enters the prompt, the prompt does not grow with the documents held: across all 830 questions its median was 21k tokens and its maximum 107k, against the model's 272k-token input limit. If it did approach the limit, the history management studied in Section V (trimming or summarising) could be added; on the reference agent, neither helped. An *initial search* with the question, counted against the cap, gives every run the same start. The turn limit is 30, and a question still running after 90 minutes counts as unanswered. Fig. 2 shows one question end to end.

So that no agent can look answers up on the web, egress is closed: the REPL refuses network modules, file access, `eval` and `exec`, and the process resolves only the model endpoint and the local retrieval server.

D. Tool-Store Agent

For the ablations, the *tool-store agent* keeps our store but replaces code with tools. It is a function-calling agent with our retriever, search cap and initial search. Its `search` tool puts whole documents into a store outside the prompt and returns one line per hit; the agent reads the store only through a `read` tool (a slice of a document) and a `grep` tool. It chooses its own width, or is held to 25 documents per search.

IV. Experimental Setup

Benchmarks. BrowseComp-Plus [4] fixes a corpus of 100,195 web pages for 830 hard multi-constraint questions from BrowseComp [29] and labels, for every question, the *gold documents* that contain the answer. These labels let us tell an agent that never found the evidence from one that found it and misread it, which makes BrowseComp-Plus our controlled testbed. FRAMES [2] (824 multi-hop questions over Wikipedia) and three ranking tasks of OBLIQ-Bench [11], whose queries describe a latent property with little word overlap, test how far the architecture carries.

Models. GPT-5 [10] serves every call, including the snippet REPL agent's sub-model, at high reasoning effort in every 150-question run, the held-out reference run and both cap-20 full-set runs; the other full-set runs used the setting recorded with each run. To test the architecture without a frontier model, we also run the open-weight Qwen3.6-35B-A3B [12]. On OBLIQ-Bench we use GPT-5.2 [30], the model of the benchmark's own agent.

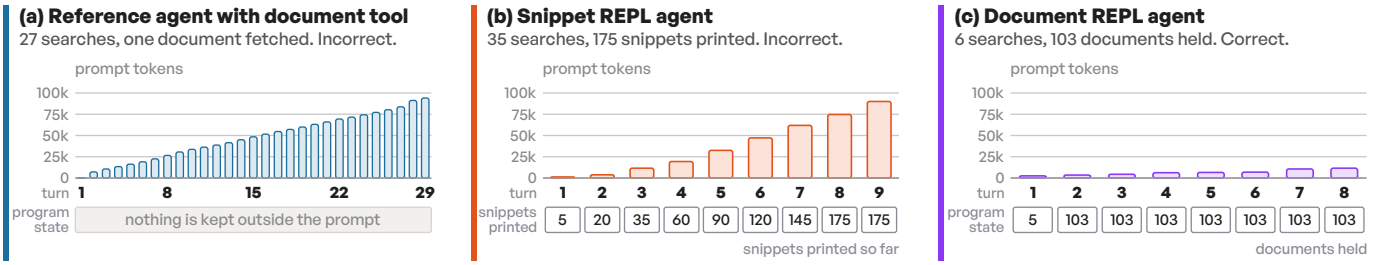


Fig. 2. One question of the 150 for three agents. Each bar is the prompt the model received at one turn (one model call; turns numbered along the x axis), on one scale; below, documents held in program state, or for the snippet REPL agent the snippets printed so far. The reference agent with the document tool (27 searches, 94k-token prompt) and the snippet REPL agent (175 snippets) answered incorrectly; the document REPL agent held 103 documents after six searches and answered correctly with an 11.7k-token final prompt. None of the three returned a labelled gold document for this question.

Retrieval. Every agent on the 150 questions uses exact dense (nearest-neighbour) search over the benchmark's Qwen3-Embedding-8B embeddings [31]. The full-set runs add BM25, a lexical retriever [32], and Reason-ModernColBERT, a late-interaction retriever [33]; OBLIQ-Bench uses Gemini-Embedding-2 [34].

Development and held-out questions. The controlled comparisons and ablations use 150 questions drawn at random, stratified by how often six earlier full-set runs of the two REPL designs had solved each. They are the development set for the REPL designs; the reference agent was run as published. The other 680 questions were not used for tuning, and the main comparison is repeated on them.

Metrics. *Accuracy*, judged by GPT-4.1 with the benchmark's template, majority of three votes; an unfinished question counts as wrong. *Gold recall*: the share of gold documents any search returned, read or not (for snippet agents a returned snippet counts). *Reading*: accuracy on the questions where all gold documents were returned. *Prompt*: tokens of the last model input. *State*: tokens held outside the prompt. *Tokens*: summed over a question's model calls.

Statistics. Agents are compared on the same questions with the exact McNemar test [35]; Table II compares our agent with each alternative; comparisons not involving our agent are reported in the text. Repeated runs of the same configuration differ by up to seven questions (about five points), and a single accuracy on 150 questions has a 95% interval of about ± 7 points, so we rely on paired tests. Holm's correction [36] over 18 comparisons on the 150 questions (the ten in Table II, four in Section V-B and four second runs) keeps those marked †; the others are exploratory. For the held-out comparison we also report how many judge verdicts would have to flip against our agent before it loses significance.

Distractor test. To separate prompt length from question difficulty, GPT-5 answers each of the 150 questions in one call given every gold document (each cut to 12k tokens, 40k in total), alone and with 25k to 200k tokens of the retriever's highest-ranked non-evidence pages added.

Other runs. The Table III runs (all 830 BrowseComp-Plus questions and FRAMES) use each agent's own configuration; their caps are not matched. The cap of 20 was chosen on 50 of the 150 questions. Table I's cap-20 row is the 150-question slice of the full-set run, judged separately; the two judgements differ on one question (90.7 versus 90.0%). The document REPL agent's cap-10 full-set runs used an earlier version of its harness with one more tool, `get_document`. We ran Fetch-then-Explore [19] with its released harness and defaults on all 830 questions: GPT-5, the same dense retriever, five 512-token snippets per search as in its harness's BrowseComp-Plus setting, no search cap, a 300-turn limit, and network access blocked except for the model endpoint. Prompts, harnesses, scripts and per-question outcomes are in our repository, without benchmark text.

V. Results

A. RQ1: Effectiveness

Tables I and II compare the document REPL agent with the reference agent and its in-prompt variants on the same 150 questions. At a cap of 10 searches it answers 83% of the 150 questions versus 69% for the reference agent. The best in-prompt variant, the reference agent with the uncapped document tool, answers 79%, 4.7 points below ours, a difference that is not significant on 150 questions. Held to the same cap of 10, every in-prompt variant falls further behind, and trimming or summarising the history does not help. Our agent also uses far fewer tokens: 185k per question against the reference agent's 895k, about a fifth, with a final prompt of 20k against 64k tokens. Its input is a sixth of the reference agent's; its output, mostly code, is 3.7 times larger.

On the 680 questions held out from development, at the tuned cap of 20, it answers 86.6% versus 77.5% for the reference agent with the document tool; the gap stays significant even if 37 judge verdicts are flipped against our agent. Fetch-then-Explore [19], which also keeps fetched documents out of the prompt but reads them through tools, five search results at a time, answers 78.4% on the same questions. It finds less of the gold evidence (recall 0.76 against 0.87; on 119 of the 830 questions no search returned a gold document, against 39 for ours, Fig. 4) and uses about four times our tokens: 772k per question against 197k, with a median of 326k against 145k.

Answer to RQ1. The architecture scores higher than the reference agent and every in-prompt variant on the 150 devel-

TABLE I

EVERY CONFIGURATION ON THE SAME 150 QUESTIONS (GPT-5, DENSE RETRIEVAL), GROUPED BY AGENT. COLUMNS AS DEFINED IN SECTION IV; READING (N): ACCURACY ON THE N QUESTIONS WITH EVERY GOLD DOCUMENT RETURNED; SNIPPET REPL TOKENS INCLUDE THE SUB-MODEL. BOLD: BEST ACCURACY AND READING.

Configuration	Acc.%	Searches	Recall	Reading% (n)	Prompt	State	Tokens in / out
Reference agent and in-prompt variants							
Reference agent	69.3	21.1	0.80	83 (107)	63.5k	–	885k / 10k
+ document tool	78.7	15.3	0.78	95 (104)	48.3k	–	696k / 9k
+ document tool, cap 10	70.0	8.6	0.67	**98** (85)	35.7k	–	255k / 9k
+ document tool, cap 10, 25 snippets	75.3	7.5	0.76	95 (100)	127.9k	–	672k / 7k
History trimmed at 30k	62.0	35.1	0.77	79 (106)	14.6k	–	420k / 13k
History summarised at 30k	71.3	26.6	0.82	85 (112)	29.0k	–	639k / 23k
Snippet REPL agent							
No cap	78.0	25.7	0.79	91 (101)	24.1k	4k	148k / 24k
Cap 10	72.7	8.3	0.62	95 (75)	15.5k	1k	44k / 10k
No cap, 25 snippets	74.7	24.7	0.88	83 (118)	26.7k	4k	179k / 18k
Fetch-then-Explore							
No cap	78.0	14.8	0.75	95 (97)	35.1k	3k	610k / 39k
Document REPL agent (ours)							
Cap 10	83.3	8.1	0.78	**98** (103)	19.9k	638k	149k / 36k
Cap 20	**90.7**	13.4	0.88	**98** (122)	20.7k	828k	158k / 35k
Ablations of ours (cap 10)							
No grep tool	82.7	8.0	0.76	97 (98)	19.6k	608k	189k / 41k
$k = 5$	70.0	8.8	0.62	96 (80)	21.1k	108k	153k / 43k
$k = 5$, documents in the prompt only	71.3	7.7	0.64	95 (79)	79.2k	–	257k / 11k
Tool store, k chosen	74.7	7.3	0.75	92 (93)	23.6k	237k	235k / 8k
Tool store, $k = 25$	78.7	7.2	0.77	96 (103)	24.9k	596k	254k / 9k
No retrieval							
Closed book	18.7	0	–	–	–	–	0.2k / 15k

TABLE II

OUR AGENT COMPARED WITH EACH ALTERNATIVE ON THE SAME QUESTIONS, EXACT MCNEMAR TEST. DIFF.: OUR ACCURACY MINUS THE ROW'S, IN POINTS. †: SIGNIFICANT AFTER HOLM'S CORRECTION (SECTION IV).

Compared with	Acc.%	Diff.	p
150 questions; ours, cap 10: 83.3%			
Reference agent	69.3	+14.0	$< 0.001^{\dagger}$
+ document tool	78.7	+4.7	0.17
+ document tool, cap 10	70.0	+13.3	$< 0.001^{\dagger}$
+ doc. tool, cap 10, 25 snippets	75.3	+8.0	0.043
History summarised	71.3	+12.0	0.005
Snippet REPL agent	78.0	+5.3	0.18
680 held-out questions; ours, cap 20: 86.6%			
Reference agent + document tool	77.5	+9.1	< 0.001
Fetch-then-Explore	78.4	+8.2	< 0.001
Ablations of ours (150 questions, cap 10)			
$k = 5$ per search	70.0	+13.3	$< 0.001^{\dagger}$
Tool store, k chosen	74.7	+8.7	0.015
Tool store, $k = 25$	78.7	+4.7	0.14
No `grep_docs`	82.7	+0.7	1

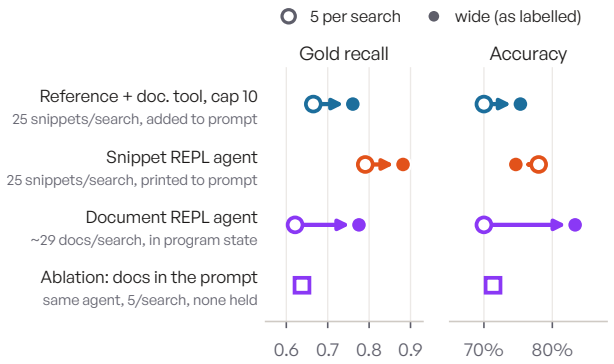


Fig. 3. Searching wider (150 questions): gold recall and accuracy of each agent at 5 documents or snippets per search (hollow) and at a wide setting (filled; the arrow runs from narrow to wide). Wide: 25 snippets per search for the capped reference agent with the document tool and for the snippet REPL agent, whose extra text enters the prompt; a mean of 28.8 documents chosen by the document REPL agent, whose extra text stays in program state. Square: an ablation of the document REPL agent at 5 per search with the documents printed into the prompt instead of held. Second runs: document REPL agent 72.0% at 5 per search and 86.7% wide; snippet REPL agent 78.0% wide.

opment questions, significantly so against the reference agent and its capped variant, and on the 680 held-out questions it answers 87% versus 78% for both the reference agent with the document tool and Fetch-then-Explore, with about a quarter of Fetch-then-Explore's tokens. On the development questions it uses about a fifth of the reference agent's tokens.

B. RQ2: Mechanism

Finding or reading. The three main agents end with similar gold recall, 0.78 to 0.80 (for the snippet agents a returned snippet counts as found), but they get there differently: the reference agent through some twenty narrow searches and a growing prompt, ours through about eight wide searches and a short prompt. The difference shows in reading: on

the questions where every gold document was returned, the reference agent answers 83%, the snippet REPL agent 91% and ours 98%, and on its longest prompts the reference agent's reading falls to 29% (n = 17). Length alone explains part of that: in the distractor test, up to 200k tokens of distractor text cost GPT-5 at most 11 points, significant only from 100k tokens on.

Width. Held to five documents per search (Fig. 3), our agent's gold recall falls from 0.78 to 0.62 and its accuracy from 83% to 70–72% (two runs), the reference agent's level. Its reading stays at 96%. Width matters for finding, not for reading, as Pi-Serini found for recall with deeper rankings [24]. Width placed in the prompt pays less (exploratory): with 25 snippets per search the capped reference agent gains five points from a 128k-token prompt, and the snippet REPL agent gains recall but not accuracy.

Where the documents live. With every retrieved document printed into the prompt and none held, at five per search, the agent answers as many questions as with the documents in program state (71.3 versus 70.0%, $p = 0.84$), but its median prompt grows from 21k to 79k tokens and exceeds the model's 272k-token input limit on 10 questions. At its usual width the agent holds a median 638k tokens, more than twice that limit.

Code or tools. The tool-store agent keeps our store but reads it through `read` and `grep` tools. Free to choose its width, it keeps its searches narrow, holding a median 41 documents versus our 105, and answers 75%. Made to take 25 documents per search, it finds as much as ours and answers 79%, 4.7 and 8.0 points behind our two runs, neither significant after correction; programmatic search agents likewise led tool agents on four of five models [23]. Only the document REPL agent chose to search wide on its own.

The other components. The `grep_docs` tool is not needed: without it the agent answers 83%, searching the held documents with Python instead. The cap sets how much the agent retrieves: raising it from 10 to 20 adds 11 questions on the 150 ($p = 0.013$) through recall (0.78 to 0.88), and 4 points on all 830 questions versus an earlier cap-10 run (Table III). History management is no substitute: deleting old results costs the reference agent 7 points (62.0 versus 69.3%, $p = 0.019$), and summarising them only recovers that loss (71.3%; $p = 0.003$ against trimming, significant after Holm's correction).

Answer to RQ2. The agents find about as much evidence but read it differently. Within the architecture, wide searches find the evidence, and keeping documents out of the prompt is what makes such searches affordable; at five documents per search, where the documents live made no detectable difference, and reading them in code rather than through tools added 4.7 to 8.0 points we could not resolve.

C. RQ3: Generality

With GPT-5, the document REPL agent leads the snippet REPL agent on BrowseComp-Plus with dense retrieval and on FRAMES (Table III). With late interaction it also leads, though not significantly. With BM25 on BrowseComp-Plus it falls behind. Fig. 4 breaks down the remaining errors: with dense and late-interaction retrieval the document REPL agent has fewer wrong answers than the snippet REPL agent in every category; with BM25 most of its errors are questions where it found no gold document (171) or gave no answer (121), and it used all 30 turns on 169 questions.

With the open-weight Qwen3.6 the architecture runs end to end and answers 44 and 58% with dense and late-interaction retrieval, against 83 and 87% with GPT-5 at the same cap, and with BM25 it scores slightly below the snippet REPL agent.

On OBLIQ-Bench (Table IV) the limit is first-stage retrieval, as its authors found [11]: in our run logs either design's searches returned only 14 to 32% of the gold documents, and the benchmark's own agent leads on two of the three tasks. Wide searches cannot recover what the retriever never surfaces.

Answer to RQ3. With GPT-5 the architecture also leads on FRAMES and, not significantly, with late-interaction retrieval, and it runs with an open-weight model at lower accuracy. It does not help where the bottleneck lies elsewhere: in first-stage retrieval on OBLIQ-Bench and in BM25 retrieval on BrowseComp-Plus (Fig. 4).

VI. Discussion

Where it does not help. When retrieval is the bottleneck, as on OBLIQ-Bench, searching wide over a weak ranking adds little; with BM25 on BrowseComp-Plus the agent often ran out of turns or gave no answer (Section V-C); and a smaller open-weight model runs the architecture but trails the frontier model by 20 to 40 points.

Design guidance.

- *Keep retrieved documents out of the prompt* and return one line per search. Read them with code or, where model-written code cannot be isolated, through `read` and `grep` tools, at a cost of 4.7 to 8.0 points we could not resolve.
- *Let the model search wide*, and fix the width if it does not choose to; bound retrieval with a search cap rather than the context window.
- *Choose the cap on a sample of questions* and check it on others.
- *If text must leave the prompt for good, summarise rather than delete.*

Limitations. Against the standard design our comparisons are between whole agents, which also differ in harness and prompt. The controlled comparisons use one model and 150 development questions, mostly in single runs; only the main comparison is repeated on held-out questions. Wide searches with whole documents in the prompt were not run, since they exceed the input limit; the location result therefore holds only at five documents per search. The capped reference variants learned the cap only when refused and had no initial search, and our snippet baselines show each document's first 512 tokens, without reranking [37]. We did not optimise how the agent searches: it writes its own queries and chooses its width with a hand-written prompt, and about half of its remaining errors are questions where it never retrieved all gold documents (Fig. 4). Natural next steps are systematic prompt

TABLE III

ACCURACY (%) ON ALL 830 BROWSECOMP-PLUS AND 824 FRAMES QUESTIONS WITH GPT-5 AND THE OPEN-WEIGHT QWEN3.6-35B-A3B. SNIPPET REPL AGENT UNCAPPED ON BROWSECOMP-PLUS WITH GPT-5 AND CAPPED AT 25 SEARCHES OTHERWISE; CAP 20 IS THE TUNED SETTING. *p*: PAIRED TEST OF THE SNIPPET REPL AND CAP-10 DOCUMENT REPL COLUMNS. PUBLISHED: THE BENCHMARK'S GPT-5 REFERENCE AGENT [4]. BOLD: BEST OF THE THREE AGENT COLUMNS.

Model	Benchmark	Retriever	Snippet REPL	Document REPL, cap 10	Document REPL, cap 20	p	Published
GPT-5	BrowseComp-Plus	dense	79.4	83.1	**87.2**	0.012	70.1
GPT-5	BrowseComp-Plus	late interaction	85.8	87.5	**88.7**	0.23	–
GPT-5	BrowseComp-Plus	BM25	**67.5**	60.0	–	< 0.001	55.9
GPT-5	FRAMES	BM25	76.3	**80.7**	–	< 0.001	–
Qwen3.6	BrowseComp-Plus	dense	–	**44.2**	–	–	–
Qwen3.6	BrowseComp-Plus	late interaction	–	**58.0**	–	–	–
Qwen3.6	BrowseComp-Plus	BM25	**24.9**	22.4	–	–	–
Qwen3.6	FRAMES	BM25	**62.0**	61.0	–	–	–

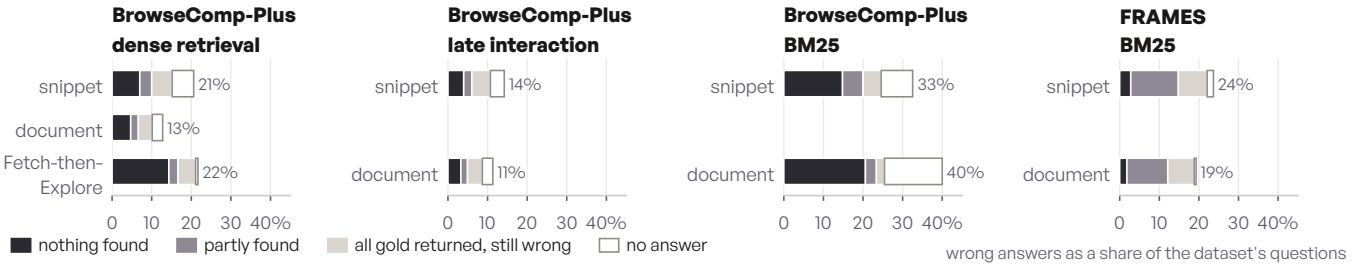


Fig. 4. Failure modes on the full sets (GPT-5): every wrong answer as a share of the questions, for the snippet REPL agent (snippet), the document REPL agent (document; cap 20 with dense and late-interaction retrieval, cap 10 with BM25) and, with dense retrieval, Fetch-then-Explore. Nothing found: no gold document was ever returned; partly found: some but not all; all gold returned, still wrong: every gold document returned, inspected or not; no answer: unfinished or abstained.

TABLE IV

OBLIQ-BENCH WITH GPT-5.2 AND GEMINI-EMBEDDING-2: THE TWO DESIGNS (20 RESULTS PER SEARCH) AND THE BENCHMARK'S OWN AGENT (OBLIQ AGENT) [11], ON ITS GOLD JUDGEMENTS (×100). BOLD: BEST PER ROW.

Task	Metric	Snippet REPL	Document REPL	OBLIQ agent
Math (151)	nDCG@10	14.9	15.9	**16.1**
	Recall@10	14.1	13.6	**14.5**
Twitter (281)	nDCG@10	10.2	9.5	**14.1**
	Recall@10	10.6	9.5	**14.1**
Congress (254)	nDCG@10	**18.8**	13.3	18.3
	Recall@10	**20.1**	15.0	18.5

optimisation such as GEPA [38] and training the search policy on the agent's explicit trajectories. Accuracy comes from a large language model (LLM) judge, and the model may have seen the corpus: without tools it answers 19% of the 150 questions.

VII. CONCLUSION

We proposed an architecture for deep-search agents that moves retrieved documents out of the prompt and into the program, where the model reads them with code and a search cap bounds retrieval. In controlled comparisons it answered more questions than the standard function-calling agent, its in-prompt variants and Fetch-then-Explore, a recent workspace-based agent, also on questions held out from development, with about a fifth of the reference agent's tokens and a quarter of Fetch-then-Explore's per question, and ablations traced the gain to the wide searches the architecture makes affordable. Keeping documents outside the prompt is what makes wide searches affordable, and the REPL is what made the model use them: with the same store read through tools, GPT-5 kept its searches narrow and answered fewer questions, and even when made to search as wide it trailed by 4.7 to 8.0 points, a gap we could not resolve. It runs with an open-weight model, which makes distilling or fine-tuning its behaviour into smaller models a natural next step.

REFERENCES

[1] Y. Huang, Y. Chen, H. Zhang, K. Li, H. Zhou, M. Fang, L. Yang, X. Li, L. Shang, S. Xu, J. Hao, K. Shao, and J. Wang, "Deep research agents: A systematic examination and roadmap," arXiv:2506.18096, 2025.

[2] S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui, "Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation," in *Proc. NAACL-HLT*, 2025, pp. 4745–4759.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[4] Z. Chen, X. Ma, S. Zhuang, P. Nie, K. Zou, S. Sharifymoghaddam, A. Liu, J. Green, K. Patel, R. Meng, M. Su, Y. Li, H. Hong, X. Shi, X. Liu, H. Oyarhoseini, N. Thakur, C. Zhang, L. Gao, W. Chen, and J. Lin, "BrowseComp-Plus: A fair and disentangled evaluation benchmark for deep search agents," in *Proc. ACL*, Jul. 2026, pp. 22 349–22 370.

[5] K. Hong, A. Troynikov, and J. Huber, "Context rot: How increasing input tokens impacts LLM performance," Chroma Technical Report, https://www.trychroma.com/research/context-rot, Jul. 2025.

[6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[7] A. L. Zhang, T. Kraska, and O. Khattab, "Recursive language models," in *Proc. NeurIPS*, 2026.

[8] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," in *Proc. ICML*, 2024, pp. 50 208–50 232.

[9] A. Jones and C. Kelly, "Code execution with MCP: Building more efficient agents," Anthropic Engineering, https://www.anthropic.com/engineering/code-execution-with-mcp, 2025.

[10] OpenAI, "OpenAI GPT-5 System Card," https://openai.com/index/gpt-5-system-card/, Aug. 2025.

[11] D. Tchuindjo, D. Shah, and O. Khattab, "OBLIQ-Bench: Exposing overlooked bottlenecks in modern retrievers with latent and implicit queries," arXiv:2605.06235, 2026.

[12] Qwen Team, "Qwen3.6-35B-A3B," Model card, https://huggingface.co/Qwen/Qwen3.6-35B-A3B, Apr. 2026.

[13] Anthropic, "Increase web search accuracy and efficiency with dynamic filtering," Claude blog, https://claude.com/blog/improved-web-search-with-dynamic-filtering, 2026.

[14] W. Cao, X. Yin, B. Dhingra, and S. Zhou, "Coding agents are effective long-context processors," arXiv:2603.20432, 2026.

[15] Z. Li, H. Zhang, C. Wei, P. Lu, P. Nie, Y. Lu, Y. Bai, S. Feng, H. Zhu, M. Zhong, Y. Zhang, J. Xie, Y. Choi, J. Zou, J. Han, W. Chen, J. Lin, D. Jiang, and Y. Zhang, "Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction," arXiv:2605.05242, 2026.

[16] Y. Lu, Z. Li, P. Nie, H. Zhang, Y. Zhang, K. Zou, W. Chen, J. Lin, D. Jiang, and Y. Zhang, "Dr-DCI: Scaling direct corpus interaction via dynamic workspace expansion," arXiv:2606.14885, 2026.

[17] Q. Liu, J. Mao, F. Zhu, T.-S. Chua, and J.-R. Wen, "OpenSaC: A Python library for search as code in deep research agents," arXiv:2610.25779, 2026.

[18] Y. Lin, E. Ang, E. Zhu, B. Ding, and J. Zhou, "Context as an environment: Programmatic context management for long-horizon agents," arXiv:2608.21690, 2026.

[19] Q. Liu, Y. Chen, Z. Chen, Y. Gao, Y. Wu, Y. Hu, J. Mao, F. Zhu, and T.-S. Chua, "Fetch-then-explore: Decoupling selection from extraction over a persistent workspace for search agents," arXiv:2608.02097, 2026.

[20] Tevatron, "BrowseComp-Plus leaderboard," https://huggingface.co/spaces/Tevatron/BrowseComp-Plus, 2026.

[21] S. Zhuang, Y. Ni, H. Fun, J. Lin, and X. Ma, "Towards retrieving interaction spaces for agentic search," arXiv:2606.06880, 2026.

[22] S. Sen, A. Kasturi, E. Lumer, A. Gulati, and V. K. Subbiah, "Is grep all you need? How agent harnesses reshape agentic search," arXiv:2605.15184, 2026.

[23] J. Qian, H. Yang, M. Liu, J. Liu, W. Shen, P. Zhou, J. Jin, J. Ma, D. Ye, and C. Chen, "Programmatic search agents: Extending agentic search beyond query reformulation," arXiv:2610.06689, 2026.

[24] T.-H. Hsu, J.-H. Yang, and J. Lin, "Rethinking agentic search with Pi-Serini: Is lexical retrieval sufficient?" arXiv:2605.10848, 2026.

[25] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. B. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng, "Context length alone hurts LLM performance despite perfect retrieval," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 23 281–23 298.

[26] X. Wu, K. Li, Y. Zhao, L. Zhang, L. Ou, H. Yin, Z. Zhang, X. Yu, D. Zhang, Y. Jiang, P. Xie, F. Huang, M. Cheng, S. Wang, H. Cheng, and J. Zhou, "ReSum: Unlocking long-horizon search intelligence via context summarization," arXiv:2509.13313, 2025.

[27] H. Zhang, Q. Xu, Z. Li, L. Zhang, P. Jiang, Y. Zhang, and J. McAuley, "Masking stale observations helps search agents – until it doesn't: A regime map and its mechanism," arXiv:2606.00408, 2026.

[28] A. L. Zhang, "rlm: General plug-and-play inference library for recursive language models," GitHub repository, https://github.com/alexzhang13/rlm, 2025.

[29] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese, "BrowseComp: A simple yet challenging benchmark for browsing agents," arXiv:2504.12516, 2025.

[30] OpenAI, "Introducing GPT-5.2," https://openai.com/index/introducing-gpt-5-2/, Dec. 2025.

[31] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv:2506.05176, 2025.

[32] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[33] A. Chaffin, "Reason-ModernColBERT," Hugging Face model card, https://huggingface.co/lightonai/Reason-ModernColBERT, 2025.

[34] M. Choi and T. Duerig, "Gemini Embedding 2: Our first natively multimodal embedding model," Google blog, https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-embedding-2/, Mar. 2026.

[35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[36] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[37] S. Sharifymoghaddam and J. Lin, "Rerank before you reason: Analyzing reranking tradeoffs through effective token cost in deep search agents," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 25 874–25 886.

[38] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," in *Proc. ICLR*, 2026.